

IntelliScale CTL Workflow

Bug Analysis & Recommendations Report

Date:	March 19, 2026
Scope:	ctl_workflow recall/reweigh bug investigation
Affected Component:	IntelliScale Django App + Odoo receiving module
Severity:	CRITICAL - Data integrity risk (0-mass bales marked as laid)

Executive Summary

The **ctl_workflow** process in IntelliScale correctly handles the initial weighing of bales and creation of delivery notes. However, when a delivery note needs to be **recalled** (for reweighing), a critical race condition between the **Celery sync task** and the **recall operation** causes the delivery note to be prematurely set to **"laid"** status in Odoo with all bales still at **0 mass**. This report identifies **7 bugs** in the recall/reweigh flow, with the race condition being the root cause of the reported issue.

The core problem: **sync_single_delivery_note()** reads the delivery note from the database, modifies odoo_data and status fields, then calls **save()** which writes ALL model fields back. If a recall operation resets scanned_bales_count to 0 between the sync's read and save, the sync **overwrites the reset**, restoring the old count. The **check_completed_delivery_notes** Celery task then sees what appears to be a fully-scanned delivery note with state="checked" and fires, setting Odoo to "laid".

Table of Contents

1. Workflow Descriptions
2. CTL Workflow Flowchart - Normal Flow
3. CTL Workflow Flowchart - Recall Flow (with bugs)
4. CTL Commercial Workflow Flowchart
5. Background Tasks Interaction Diagram
6. Bug Registry (7 bugs identified)
7. Root Cause Analysis
8. Recommendations & Fixes
9. Files Affected

1. Workflow Descriptions

1.1 ctl_workflow (Grower Workflow)

The **ctl_workflow** is the primary process for weighing grower tobacco bales. It requires delivery notes to be in "**checked**" state before scanning can begin. The flow is:

1. Odoo creates delivery notes with bales (synced to IntelliScale every 30s)
2. Delivery note must be in "checked" state (verified by receiving staff in Odoo)
3. Operator scans a bale barcode at the weighing station
4. System finds the matching delivery note, activates it for scanning
5. Scale weight is captured and sent to Odoo via `/api/bales/update-mass/`
6. Barcode is added to the scanned list, count incremented
7. Repeat until all bales are scanned (`is_scanning_complete() == True`)
8. Operator explicitly clicks "Close Delivery Note" to set Odoo state to "laid"

1.2 ctl_commercial_workflow (Commercial Workflow)

The **ctl_commercial_workflow** is similar but designed for commercial/marshalled bales. Key differences:

- **No state check:** Does NOT require state="checked" before scanning
- **allow_bale_insert:** Can create new bales in Odoo via `/api/bales/create-commercial-bale`
- **allow_marshallling:** Supports `lot_number` and `group_number` marshalling fields
- **Different close endpoint:** Uses `/api/grower-delivery-notes/close-marshalled-at-scale`
- Uses **number_of_bales_delivered** instead of `number_of_bales` for completion tracking

1.3 Recall Mechanisms

There are three recall mechanisms available:

Mechanism	Endpoint	Odoo Effect	Django Effect
Full DNote Recall (recall_delivery_note)	<code>/api/bales/recall-all-bale</code>	All masses -> 0 State -> 'checked'	Records deleted scanned = 0
Single Bale Recall (recall_bale view)	<code>/api/bales/recall-bale/</code>	Bale mass -> 0 DNote state unchanged	Record deleted count -= 1
Recall & Update (recall_and_update_bale)	<code>/api/bales/update-mass/</code> (mass=0.01)	Bale mass -> 0.01 DNote state unchanged	Record deleted count -= 1

2. CTL Workflow - Normal Flow (New Bales)

This flowchart shows the normal scanning flow for new bales. This flow works correctly.

CTL WORKFLOW (ctl_workflow) - New Bales Flow

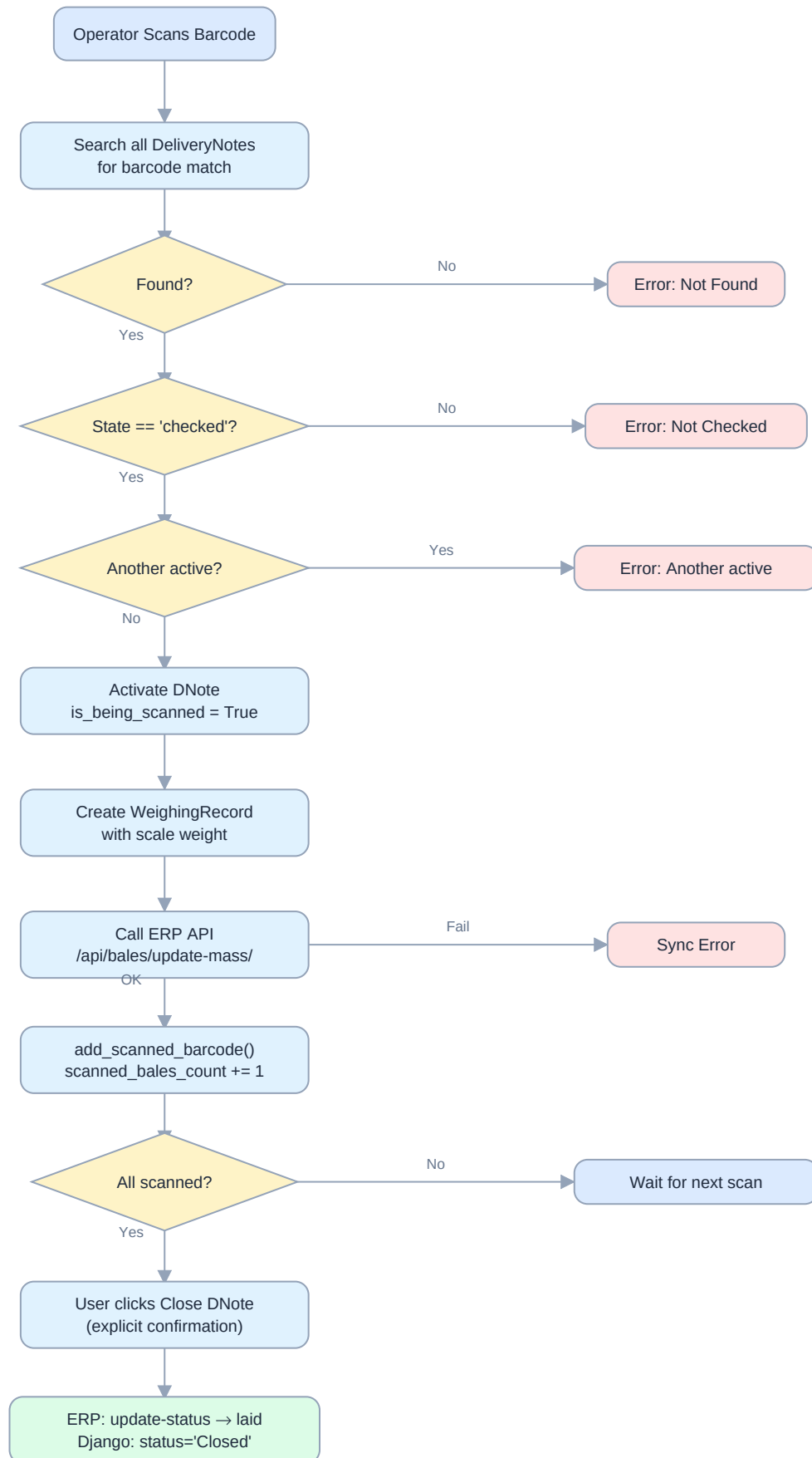


Figure 1: CTL Workflow - Normal bale scanning flow

3. CTL Workflow - Recall & Reweigh Flow (WITH BUGS)

This flowchart shows what happens when a delivery note is recalled for reweighing. The red annotations highlight the race condition that causes the reported bug.

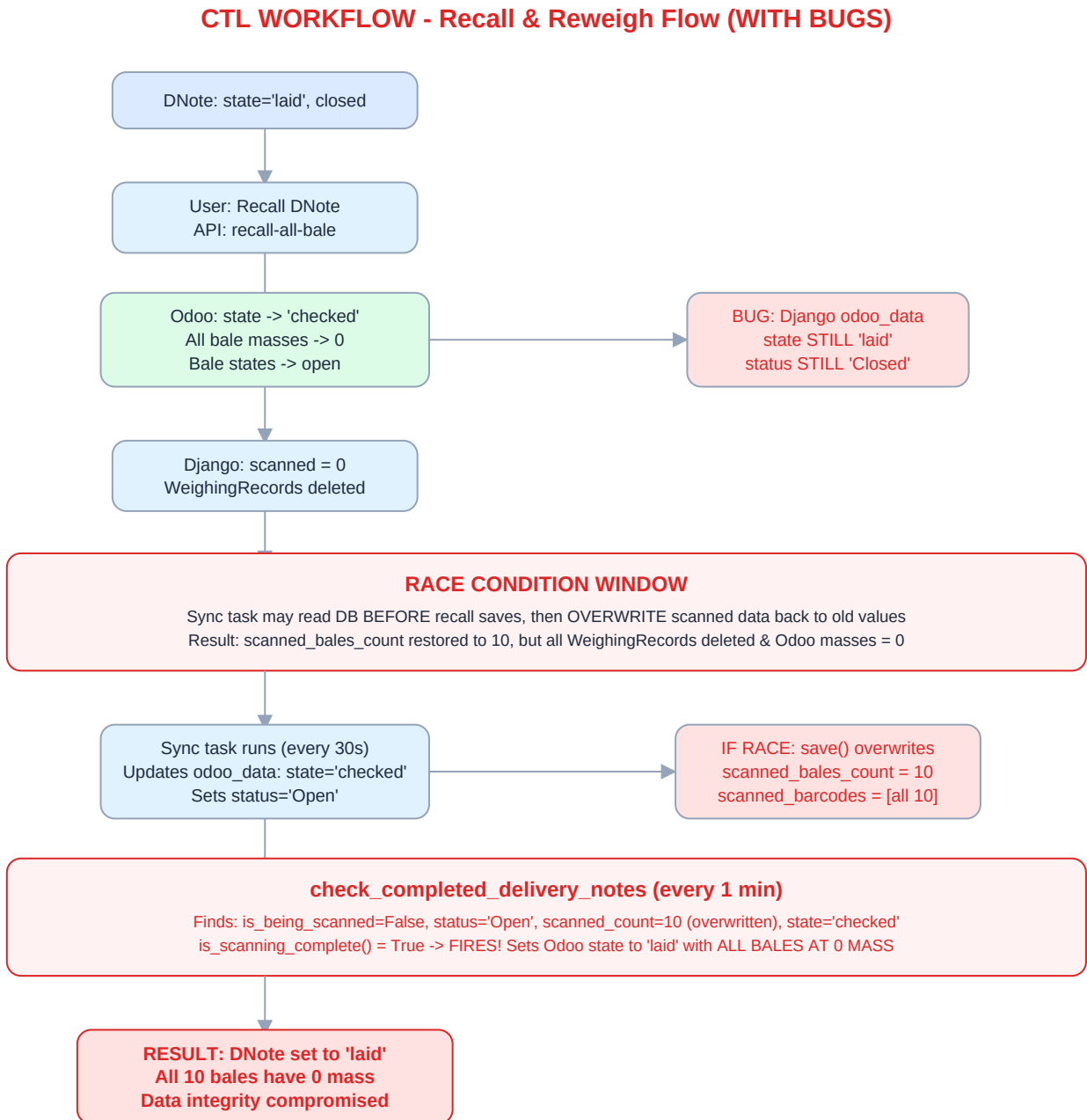


Figure 2: CTL Workflow - Recall flow showing race condition bug

4. CTL Commercial Workflow

The commercial workflow differs primarily in not requiring the "checked" state and supporting bale insertion (creating new bales in Odoo during scanning).

CTL COMMERCIAL WORKFLOW (ctl_commercial_workflow)

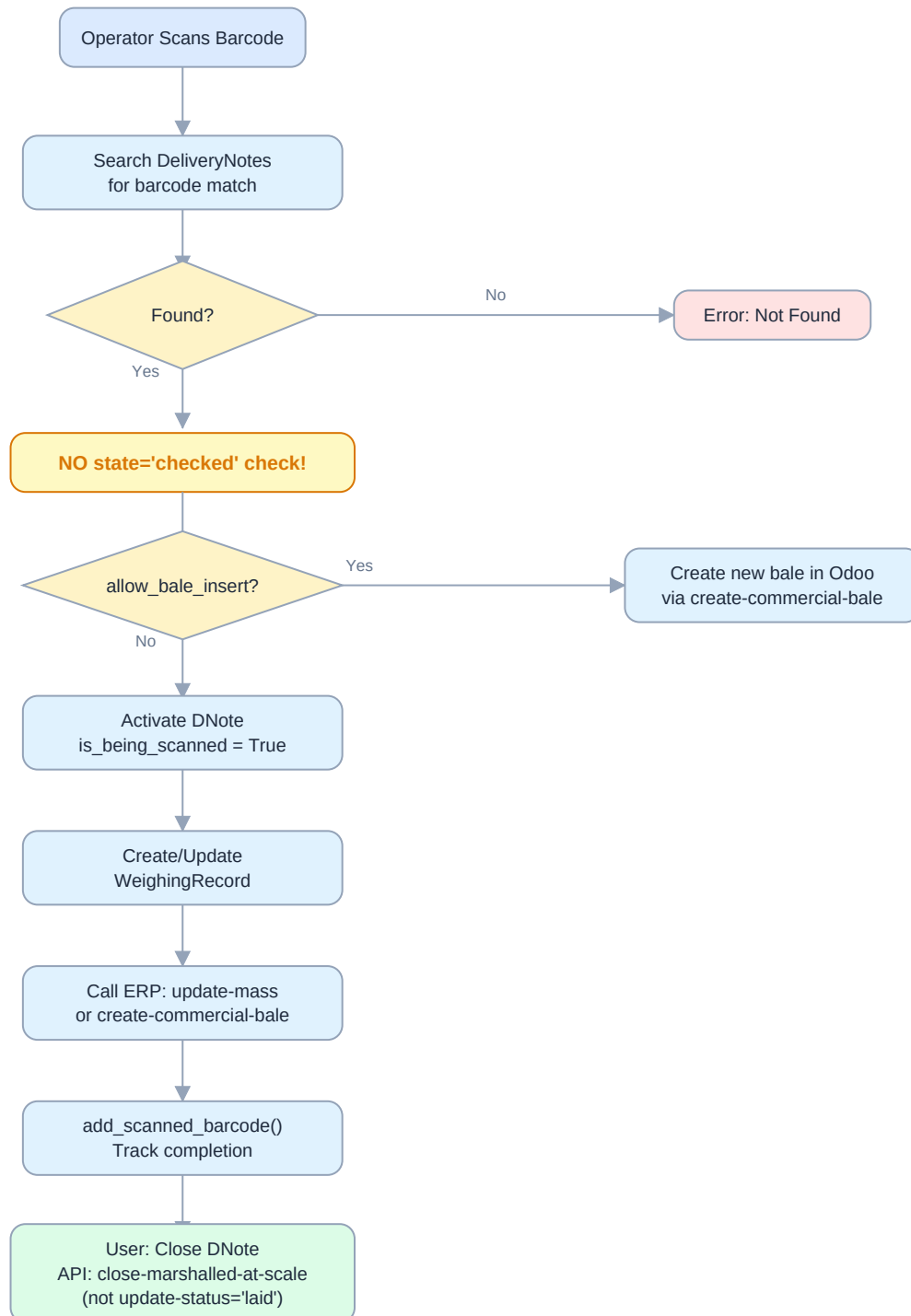


Figure 3: CTL Commercial Workflow flow

5. Background Tasks Interaction

Two Celery tasks run in the background and interact with the delivery note lifecycle. The lack of locking in the sync task is the enabler for the race condition bug.

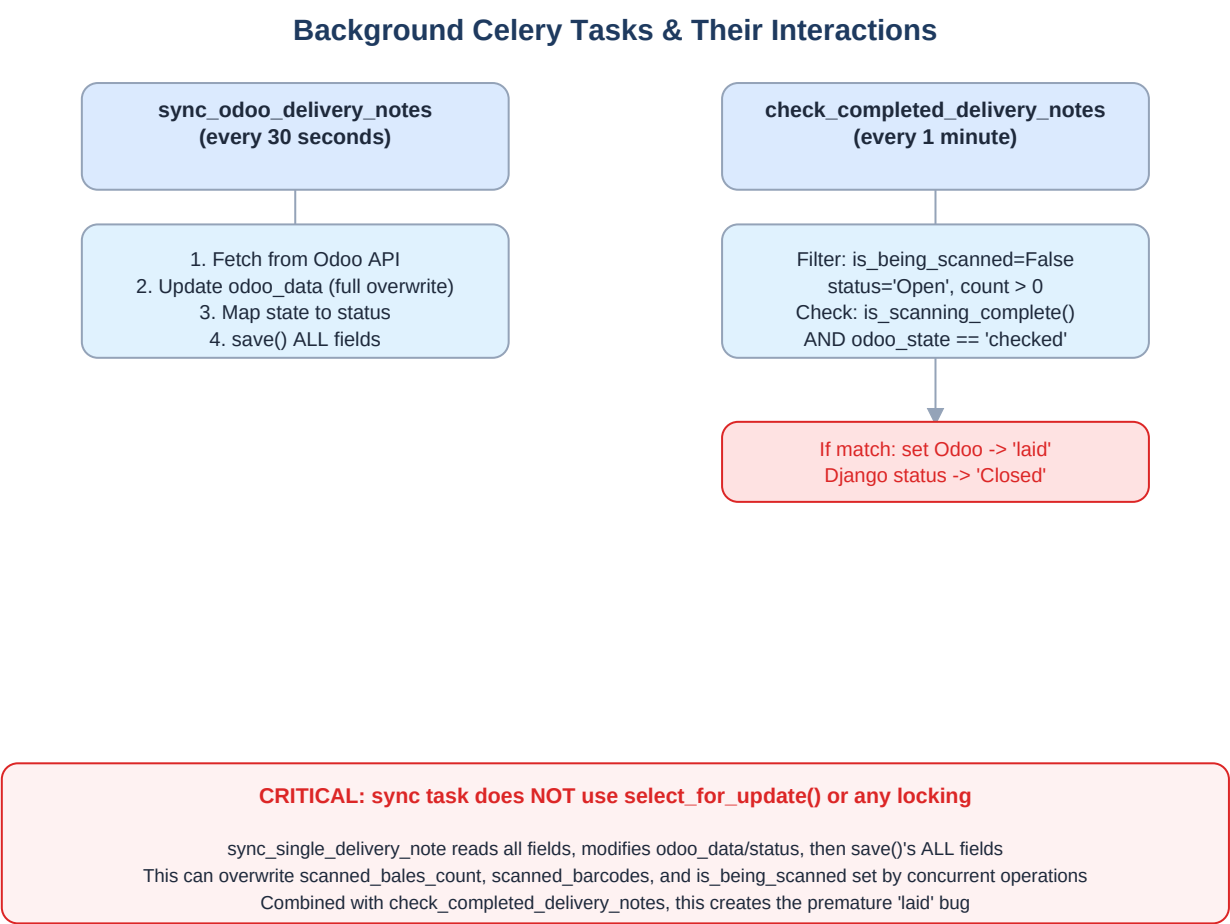


Figure 4: Background Celery tasks and their interactions

6. Bug Registry

Seven bugs were identified during the analysis. Bug #1 is the root cause of the reported issue. The remaining bugs are contributing factors or independent issues in the recall/reweigh flow.

BUG-001: Race Condition - Sync Task Overwrites Recall Data [CRITICAL]

Location: `scale/tasks.py` :: `sync_single_delivery_note()` + `scale/views/delivery_note.py` :: `recall_delivery_note()`

Root Cause: `sync_single_delivery_note()` reads ALL delivery note fields from the database, modifies only `odoo_data` and `status`, then calls `save()` which writes ALL fields back. If `recall_delivery_note()` resets `scanned_bales_count` to 0 and `scanned_barcode`s to [] between the sync's DB read and its `save()`, the sync overwrites these fields with their pre-recall values (e.g., `count=10`, `barcode`s=[all 10]).

Impact: After the overwrite, `check_completed_delivery_notes()` sees `scanned_bales_count=10` (matching `number_of_bales=10`), `odoo_state="checked"`, `is_being_scanned=False`, and `status="Open"`. All conditions are met, so it calls `update_dnote_completion_status()` which sets the Odoo state to "laid" - even though all bales have 0 mass. This is the exact bug reported by the user.

Reproduction: Trigger a delivery note recall while the sync task is running. The 30-second sync interval creates a high probability of overlap.

BUG-002: recall_delivery_note Does Not Update Local odoo_data [HIGH]

Location: `scale/views/delivery_note.py` :: `recall_delivery_note()` lines 154-159

Description: After the recall-all-bale API succeeds (setting Odoo state to "checked"), the local `odoo_data[state]` remains "laid". The `ctl_workflow` requires `state=="checked"` before scanning, so the operator cannot scan until the next sync task runs (up to 30 seconds). This creates an unnecessary delay and confusing UX where the recall appears to succeed but scanning is blocked.

Fix: After successful recall API, set `delivery_note.odoo_data[state] = 'checked'` locally.

BUG-003: recall_delivery_note Does Not Reset Local Status [HIGH]

Location: `scale/views/delivery_note.py` :: `recall_delivery_note()` lines 154-159

Description: After recall, the local status field remains "Closed" (from the original close). The delivery note does not appear in the "Open" delivery notes list on the weighing station until the sync task runs and sets `status="Open"`. Combined with BUG-002, this means the delivery note is invisible and unscalable for up to 30 seconds after recall.

Fix: After successful recall API, set `delivery_note.status = 'Open'`.

BUG-004: Sync Maps "laid" State to "Open" Status [MEDIUM]

Location: `scale/tasks.py` :: `sync_single_delivery_note()` lines 121-124

Description: The state-to-status mapping includes "laid" in the list that maps to "Open": `if odoo_record['state'] in ['open', 'checked', 'laid']: delivery_note.status = 'open'`. This means after a delivery note is legitimately closed (`state="laid"`, `status="Closed"`), the next sync resets status back to "Open". While `check_completed_delivery_notes` requires `odoo_state=="checked"` (not "laid") to fire, this creates an inconsistent local state and can interact with other bugs.

Fix: Change the mapping to exclude "laid" from the "Open" status mapping. Delivery notes with `state="laid"` should map to `status="Closed"` locally.

BUG-005: Individual Bale Recall Does Not Transition DNote State [MEDIUM]

Location: `scale/views/ajax.py` :: `recall_and_update_bale()` and `scale/views/delivery_note.py` :: `recall_bale()`

Description: When individual bales are recalled from a "laid" delivery note, the Odoo delivery note state remains "laid". The `ctl_workflow` requires `state=="checked"` to scan, so the operator cannot rescan the recalled bale. The `recall_and_update_bale` endpoint calls `/api/bales/update-mass/` with `mass=0.01` (not `/api/bales/recall-bale/`), and neither endpoint changes the delivery note state. The operator is stuck: the bale is recalled but cannot be re-weighed.

Fix: After individual bale recall on a "laid" dnote, call the Odoo API to transition the delivery note state back to "checked", or add an IntelliScale-specific endpoint that handles this transition.

BUG-006: recall_and_update_bale Uses update-mass Instead of recall-bale [LOW]

Location: `scale/views/ajax.py :: recall_and_update_bale()` line 106

Description: The `recall_and_update_bale` endpoint calls `/api/bales/update-mass/?mass=0.01` instead of `/api/bales/recall-bale/`. The `update-mass` API validates `mass > 0`, so 0.01 is used as a workaround. However, the `recall-bale` API exists specifically for this purpose and sets `mass` to exactly 0. Using `update-mass` leaves bales with `mass=0.01` in Odoo rather than 0, which could affect mass totals and reports.

Fix: Use the `/api/bales/recall-bale/` endpoint for individual bale recalls.

BUG-007: No Concurrency Protection on DeliveryNote save() [MEDIUM]

Location: `scale/tasks.py :: sync_single_delivery_note()` and `scale/models/delivery_note.py`

Description: The sync task uses a plain `get_or_create() + save()` pattern without any concurrency control. Since IntelliScale uses SQLite (which doesn't support row-level locking), even Django's `select_for_update()` in the weighing station view provides limited protection. Any concurrent modification of a delivery note during sync can lead to lost updates. This affects not just recall but also normal scanning operations that coincide with sync cycles.

Fix: Use `update()` with specific fields instead of `save()` in the sync task. Alternatively, use `F()` expressions or conditional updates to avoid overwriting fields managed by other operations.

Bug Summary

ID	Severity	Description	Status
BUG-001	CRITICAL	Race condition: sync overwrites recall data	Root Cause
BUG-002	HIGH	Recall does not update local odoo_data state	Contributing
BUG-003	HIGH	Recall does not reset local status	Contributing
BUG-004	MEDIUM	Sync maps "laid" to "Open" status	Independent
BUG-005	MEDIUM	Individual recall blocks rescan (state stuck)	Independent
BUG-006	LOW	Uses update-mass instead of recall-bale	Independent
BUG-007	MEDIUM	No concurrency protection on sync save()	Enabler

7. Root Cause Analysis

The Sequence of Events

The reported bug ("scanning the first barcode on a recalled dnote sets it to laid") is caused by the following sequence:

Step	Action	Django State	Odoo State
1	All 10 bales scanned, dnote closed	status='Closed' scanned_count=10 is_being_scanned=False	state='laid' All masses > 0
2	User triggers Recall DNote	API call to Odoo... (takes ~1-3 seconds)	Processing recall...
3	CONCURRENTLY: Sync task reads Django DB	Sync reads: scanned_count=10 scanned_barcodes=[all]	(Sync reading Odoo API)
4	Recall API returns OK. Django resets data.	scanned_count=0 scanned_barcodes=[] Records deleted	state='checked' All masses=0
5	Sync task saves (overwrites!)	scanned_count=10 (!!!) scanned_barcodes=[all] odoo_data.state='checked' status='Open'	state='checked' All masses=0
6	check_completed task runs (every 1 min)	Sees: count=10, complete, state='checked' FIRES -> set to laid	state='laid' (!!!) All masses STILL 0
7	Operator scans barcode	ERROR: state='laid' not 'checked' OR dnote already laid	CORRUPTED: 10 bales with 0 mass marked as 'laid'

Table 1: Race condition sequence of events (rows 5-7 highlighted as critical)

Why This Is Hard to Reproduce Consistently

The race condition depends on the sync task (30-second interval) reading the Django database BEFORE the recall operation saves, but completing its save AFTER. This is a ~1-3 second window during each 30-second cycle. The probability is roughly 3-10% per recall operation, making it intermittent but frequent enough to be a real production issue.

8. Recommendations

REC-001: Use `update()` Instead of `save()` in Sync Task [CRITICAL FIX]

Change `sync_single_delivery_note()` to use Django's **`update()`** method with only the fields it owns, instead of the full-object `save()` pattern. This prevents overwriting fields managed by other operations.

```
# BEFORE (BUG): reads all fields, saves all fields
delivery_note.odoo_data = odoo_record
delivery_note.status = 'Open'
delivery_note.save() # Overwrites scanned_bales_count, is_being_scanned, etc.

# AFTER (FIX): only update the fields the sync task owns
DeliveryNote.objects.filter(pk=delivery_note.pk).update(
    odoo_data=odoo_record,
    delivery_note_number=document_number,
    status=mapped_status,
    is_synced=True,
    last_sync_attempt=timezone.now(),
    sync_error_message=''
)
```

Note: Using `update()` bypasses the model's `save()` method (and thus QR code generation). QR code generation should be moved to a separate method or triggered only on creation.

REC-002: Update Local State After Recall [HIGH PRIORITY]

After a successful recall API call, immediately update the local delivery note state to match what Odoo now has. This eliminates the 30-second blind window and prevents UX confusion.

```
# In recall_delivery_note(), after successful API call:
with transaction.atomic():
    WeighingRecord.objects.filter(delivery_note=delivery_note).delete()
    delivery_note.scanned_barcodes = []
    delivery_note.scanned_bales_count = 0
    delivery_note.status = 'Open' # NEW
    delivery_note.is_being_scanned = False # NEW (explicit)
    if delivery_note.odoo_data: # NEW
        delivery_note.odoo_data['state'] = 'checked'
    delivery_note.save()
```

REC-003: Fix Status Mapping for "laid" State [MEDIUM]

Change the sync task's state-to-status mapping so that "laid" maps to "Closed", not "Open". This prevents the sync from reopening legitimately closed delivery notes.

```
# BEFORE:
if odoo_record['state'] in ['open', 'checked', 'laid']:
    delivery_note.status = 'Open'

# AFTER:
if odoo_record['state'] in ['open', 'checked']:
    mapped_status = 'Open'
else: # laid, closed, etc.
    mapped_status = 'Closed'
```

REC-004: Add State Transition for Individual Bale Recall [MEDIUM]

When individual bales are recalled from a "laid" delivery note, add logic to transition the delivery note state back to "checked" in Odoo. This enables the operator to rescan the recalled bale without needing a full delivery note recall.

Option A: Call the update-status API to set state="checked" when recalling from a "laid" dnote.

Option B: Create a new Odoo API endpoint specifically for "reweigh mode" that partially reopens a dnote.

REC-005: Use recall-bale API for Individual Recalls [LOW]

Change `recall_and_update_bale` (ajax.py) to use `/api/bales/recall-bale/` instead of `/api/bales/update-mass/?mass=0.01`. The recall-bale endpoint sets mass to exactly 0, which is semantically correct and doesn't leave stale mass values in the database.

REC-006: Add Validation Guard to check_completed_delivery_notes [DEFENSE IN DEPTH]

Add an additional check in the `check_completed_delivery_notes` task to verify that `WeighingRecords` actually exist for the scanned barcodes before setting the dnote to "laid". This provides a safety net against the race condition even before the sync task is fixed.

```
# In check_completed_delivery_notes(), before firing:
actual_records = WeighingRecord.objects.filter(
    delivery_note=dnote, is_synced=True
).count()
if actual_records < dnote.scanned_bales_count:
    logger.warning("Mismatch: scanned_count=%s but only %s records",
        dnote.scanned_bales_count, actual_records)
    continue # Skip - data may be corrupted by race condition
```

9. Files Affected

File	Changes Required
intelliscale/scale/tasks.py	sync_single_delivery_note: use update() instead of save() check_completed_delivery_notes: add WeighingRecord validation guard
intelliscale/scale/views/delivery_note.py	recall_delivery_note: update local state after recall recall_bale: add dnote state transition for "laid" dnotes
intelliscale/scale/views/ajax.py	recall_and_update_bale: use recall-bale API Add dnote state transition for "laid" dnotes
intelliscale/scale/models/delivery_note.py	Consider removing QR generation from save() (move to explicit method)
odoo-custom-apps/receiving/controllers/intelliscale.py	recall_bale: optionally transition dnote state when all bales have 0 mass

Implementation Priority

Priority	Recommendation	Fixes Bugs	Effort
1 (Do First)	REC-001: update() in sync task	BUG-001, BUG-007	Low (1-2 hours)
2	REC-002: Update local state after recall	BUG-002, BUG-003	Low (30 min)
3	REC-006: Validation guard in check task	BUG-001 (defense)	Low (30 min)
4	REC-003: Fix laid->Open status mapping	BUG-004	Low (15 min)
5	REC-004: State transition for individual recall	BUG-005	Medium (2-3 hours)
6	REC-005: Use recall-bale API	BUG-006	Low (30 min)